

Guide complet d'installation Raspberry Pi - Cycle Analyst App

Derniere mise a jour : 2026-07-05.

Ce guide decrit une installation propre d'un nouveau Raspberry Pi avec le repo `Cycle-Analyst-App`, les variables d'environnement utiles et les services `systemd`.

Le setup recommande installe :

- l'enregistreur embarque `cycle-recorder.service`
- le worker photo `cycle-photo.service`
- l'interface Cycle Analyst sur le port 5050
- un nom local stable en `.local` via mDNS
- un proxy `nginx` sur le port 80
- trois services `systemd` : `cycle-recorder.service`, `cycle-photo.service` et `cycle-analyst.service`
- un fichier de variables local dedie : `$HOME/Cycle-Analyst-App/cycle-analyst.env`
- optionnellement le `monitor_server` sur le port 8080

Sommaire

Clique sur une section pour aller directement a la bonne partie du guide.

Preparation du Raspberry Pi

1. [Hypotheses et conventions](#)
2. [Preparer la carte SD](#)
3. [Premier acces SSH](#)
4. [Verifier ou corriger le hostname](#)
5. [Mettre a jour le systeme et installer les paquets](#)

Installation de l'application

6. [Installer le repo](#)
7. [Creer le dossier runtime](#)
8. [Ajuster le fichier de variables d'environnement](#)
9. [Variables disponibles pour l'app embarquee](#)

Carte offline et peripheriques

10. [Telecharger les tiles PMTiles pour la basemap vectorielle](#)
11. [Tester l'app manuellement](#)
12. [Tester les peripheriques optionnels](#)

Services et acces reseau

- 13. Installer les services systemd separes
- 14. Autoriser le bouton de redemarrage de l'UI
- 15. Configurer nginx pour l'URL locale
- 16. Verifier le mDNS . Local
- 17. Option : creer un hotspot Wi-Fi de secours
- 18. Config SSH pour VS Code

Maintenance et monitor

- 19. Mise a jour du repo sur un Pi deja installe
- 20. Option : installer monitor_server
- 21. Brancher un Pi vehicule au monitor

Validation et depannage

- 22. Checklist finale par Pi
- 23. Depannage
- 24. Resume ultra-court

1. Hypotheses et conventions

Adapte ces valeurs a chaque Pi, puis remplace-les dans les commandes quand un exemple en dur apparait. Les deux valeurs qui changent le plus souvent sont l'utilisateur Linux et le hostname.

- utilisateur Linux : `jeandard`
- hostname : `sc-vehicule-1`
- dossier du repo : `/home/<utilisateur>/Cycle-Analyst-App`
- environnement Python : `/home/<utilisateur>/Cycle-Analyst-App/.venv`
- dossier runtime : `/home/<utilisateur>/Cycle-Analyst-App/var`
- app locale : `http://127.0.0.1:5050`
- app exposee via nginx : `http://<hostname>.local`
- service enregistrement : `cycle-recorder.service`
- service photo : `cycle-photo.service`
- service interface : `cycle-analyst.service`
- fichier env app : `/home/<utilisateur>/Cycle-Analyst-App/cycle-analyst.env`

Exemple pour le Pi client `sc-vehicule-5` :

- utilisateur Linux : `danieldilg`
- hostname : `sc-vehicule-5`
- dossier du repo : `/home/danieldilg/Cycle-Analyst-App`
- environnement Python : `/home/danieldilg/Cycle-Analyst-App/.venv`
- app exposee via nginx : `http://sc-vehicule-5.local`
- fichier env app : `/home/danieldilg/Cycle-Analyst-App/cycle-analyst.env`

Sur le Pi, tu peux verifier les valeurs actives avec :

```
whoami
hostname
pwd
```

Important :

- dans les commandes shell, tu peux utiliser `$HOME`, `$USER` et `$(hostname)`
- dans `cycle-analyst.env`, les fichiers `.service`, `nginx` et `sudoers`, mets les valeurs reelles, par exemple `/home/danieldilg/...` et `danieldilg`

Exemple de noms par vehicule :

- Pi 1 : `sc-vehicule-1`
- Pi 2 : `sc-vehicule-2`

- Pi 3 : `sc-vehicule-3`
- Pi 5 : `sc-vehicule-5`

Le nom complet sur le reseau sera ensuite :

- `sc-vehicule-1.local`
- `sc-vehicule-2.local`
- `sc-vehicule-3.local`
- `sc-vehicule-5.local`

Les services systemd gardent les memes noms sur tous les Pi. Ce qui change, c'est leur contenu : le script `scripts/setup_pi_services.py --apply` ecrit automatiquement `User=...`, `Group=...`, `WorkingDirectory=...` et le `server_name` nginx avec les valeurs du Pi courant.

2. Preparer la carte SD

Avec Raspberry Pi Imager :

1. Choisir le modele de Raspberry Pi.
2. Choisir `Raspberry Pi OS Lite (64-bit)` pour un Pi sans ecran local.
3. Choisir la carte SD.
4. Ouvrir les options avancees avant l'ecriture.
5. Definir le hostname, par exemple `sc-vehicule-1` ou `sc-vehicule-5`.
6. Activer SSH.
7. Definir l'utilisateur Linux, par exemple `jeandard` ou `danieldilg`.
8. Definir un mot de passe ou ajouter ta cle SSH publique.
9. Regler le pays Wi-Fi, la locale et le fuseau horaire.
10. Ajouter le Wi-Fi du hotspot telephone si tu veux que le Pi s'y connecte au premier boot.

Ecrire la carte SD, l'insérer dans le Pi puis demarrer.

3. Premier acces SSH

Depuis ton Mac ou ton PC :

```
ssh <utilisateur>@<hostname>.local
```

Exemple pour le Pi client :

```
ssh danieldilg@sc-vehicule-5.local
```

Si le `.local` ne repond pas encore, connecte-toi avec l'IP du Pi :

```
ssh <utilisateur>@IP_DU_PI
```

Pour afficher l'IP depuis le Pi :

```
hostname -I
```

4. Verifier ou corriger le hostname

Sur le Pi :

```
hostname
hostnamectl
cat /etc/hostname
```

Resultat attendu pour le Pi 1 :

```
- hostname : sc-vehicule-1
- hostnamectl: Static hostname: sc-vehicule-1
- /etc/hostname : sc-vehicule-1
```

Si le nom est incorrect :

```
sudo hostnamectl set-hostname sc-vehicule-5
echo 'preserve_hostname: true' | sudo tee /etc/cloud/cloud.cfg.d/99-preserve-hostname.cfg
sudo reboot
```

Remplace `sc-vehicule-5` par le hostname voulu pour le Pi courant.

Apres reboot :

```
hostname
hostnamectl
cat /etc/hostname
```

5. Mettre a jour le systeme et installer les paquets

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y git python3 python3-venv python3-pip nginx avahi-daemon curl i2c-tools
```

Installer les outils pour la camera USB :

```
sudo apt install -y fswebcam v4l-utils
```

Verifier que la camera USB est visible :

```
lsusb
v4l2-ctl --list-devices
```

Activer mDNS :

```
sudo systemctl enable --now avahi-daemon
```

```
systemctl status avahi-daemon --no-pager
```

Ajouter l'utilisateur aux groupes utiles pour les ports serie, I2C et camera :

```
sudo usermod -aG dialout,i2c,video "$USER"
```

Activer I2C si le capteur solaire INA228 est utilise :

```
sudo raspi-config nonint do_i2c 0
```

Redemarrer pour appliquer les groupes et I2C :

```
sudo reboot
```

Reconnecte-toi ensuite en SSH.

6. Installer le repo

6.1 Donner acces au repo GitHub

Le repo est :

```
https://github.com/regenbox-team/Cycle-Analyst-App.git
```

Pour un Pi client ou un Pi qui ne doit pas avoir de cle SSH GitHub, utilise HTTPS avec le compte GitHub du client et un Personal Access Token.

Le compte GitHub doit avoir un acces **Read** au repo **regenbox-team/Cycle-Analyst-App**. Au moment du clone :

- **Username** : le username GitHub qui a acces au repo, par exemple **dna2087**
- **Password** : un token GitHub, pas le mot de passe du compte

Le token doit avoir acces au repo et au minimum la permission `Contents: Read-only`. GitHub l'affiche une seule fois au moment de sa creation.

Configurer la memorisation locale du token sur le Pi :

```
git config --global credential.helper store
```

Note : **credential.helper store** stocke le token en clair dans **~/.git-credentials**. Pour un Pi client, utilise un compte/token client limite au repo.

6.2 Option SSH avec cle dediee

Sur le Pi, generer une cle dediee :

```
ssh-keygen -t ed25519 -C "sc-vehicule-1@github" -f ~/.ssh/github_sc_vehicule_1
```

Pour le Pi 2, utilise plutot :

```
ssh-keygen -t ed25519 -C "sc-vehicule-2@github" -f ~/.ssh/github_sc_vehicule_2
```

Quand `ssh-keygen` demande une passphrase, tu peux laisser vide pour un Pi qui doit faire des `git pull` sans interaction.

Afficher la cle publique :

```
cat ~/.ssh/github_sc_vehicule_1.pub
```

Copier toute la ligne affichee, qui commence par `ssh-ed25519`.

Dans GitHub :

1. Ouvrir ton profil utilisateur.
2. Aller dans `Settings` puis `SSH / GPG Keys`.
3. Cliquer sur `Add Key`.
4. Nommer la cle, par exemple `sc-vehicule-1`.
5. Coller le contenu de `~/.ssh/github_sc_vehicule_1.pub`.
6. Enregistrer.

Si tu veux limiter la cle a un seul repo, utilise plutot une Deploy Key dans le repo GitHub :

1. Ouvrir le repo dans GitHub.
2. Aller dans `Settings` puis `Deploy Keys`.
3. Ajouter la cle publique du Pi.
4. Laisser l'écriture desactivée si le Pi doit seulement cloner et pull.

Créer ensuite une config SSH locale sur le Pi :

```
nano ~/.ssh/config
```

Configuration GitHub :

```
Host github
  HostName github.com
  User git
  IdentityFile ~/.ssh/github_sc_vehicule_1
  IdentitiesOnly yes
  Port 22
```

Protéger les fichiers SSH :

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/config ~/.ssh/github_sc_vehicule_1
chmod 644 ~/.ssh/github_sc_vehicule_1.pub
```

Tester la connexion :

```
ssh -T github
```

Au premier test, SSH peut demander de confirmer l'empreinte du serveur GitHub.

Repondre `yes` seulement si c'est bien ton serveur.

Si GitHub repond avec un message du type ``Hi <user>! You've successfully authenticated``, c'est bon. GitHub precise aussi qu'il ne fournit pas de shell interactif, c'est normal.

L'URL de clone SSH ressemble ensuite a :

```
git@github.com:regenbox-team/Cycle-Analyst-App.git
```

Avec l'alias `Host github`, tu peux aussi utiliser :

```
github:regenbox-team/Cycle-Analyst-App.git
```

6.3 Cloner et installer les dependances

```
cd "$HOME"
git clone https://github.com/regenbox-team/Cycle-Analyst-App.git Cycle-Analyst-App
cd "$HOME/Cycle-Analyst-App"
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Si tu as configure l'option SSH, tu peux remplacer la commande `git clone` par :

```
git clone github:regenbox-team/Cycle-Analyst-App.git Cycle-Analyst-App
```

Si le repo est deja clone et que tu es deja dans le venv, par exemple :

```
(.venv) danieldilg@sc-vehicule-5:~/Cycle-Analyst-App
```

continue directement avec :

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
mkdir -p var var/session_metrics var/live_photo var/pending_photos
mkdir -p "$HOME/Documents"
```

6.4 Suite directe pour une installation standard

Les sections 7 a 12 detailent les dossiers, le fichier env, la carte offline, le GPS, le capteur solaire et la camera. Si tu fais une installation standard et que tu veux simplement demarrer les services, tu peux enchaîner directement avec ces commandes depuis le repo :


```
mkdir -p var var/session_metrics var/live_photo var/pending_photos
mkdir -p "$HOME/Documents"
python3 scripts/setup_pi_services.py
python3 scripts/setup_pi_services.py --apply
```

Ces deux commandes `setup_pi_services.py` sont detaillees en section 13. La premiere affiche un apercu; la seconde installe les services et cree `cycle-analyst.env` s'il n'existe pas encore.

Apres `--apply`, verifie les valeurs locales :

```
grep -nE '^(MONITOR_DEVICE_ID|APP_PMTILES_PATH)= ' cycle-analyst.env
systemctl status cycle-recorder.service cycle-photo.service cycle-analyst.service --no-pager
```

7. Creer le dossier runtime

Le code cree `var/` automatiquement, mais c'est plus clair de le preparer :

```
cd "$HOME/Cycle-Analyst-App"
mkdir -p var var/session_metrics var/live_photo var/pending_photos
```

Les fichiers runtime principaux sont :

- `var/ride_data_supercycle_live.db`
- `var/ride_data_supercycle_test.db`
- `var/session_metrics/*.json`
- `var/current_session.txt`
- `var/session_state.txt`
- `var/current_user.txt`
- `var/current_user_id.txt`
- `var/users.json`
- `var/vehicle_mode.txt`
- `var/test_mode.txt`
- `var/solar_roof.txt`
- `var/solar_battery_state.json`
- `var/route.gpx`
- `var/live_photo/*.jpg`
- `var/pending_photos/*` : photos en attente d'envoi si le reseau est indisponible.

8. Ajuster le fichier de variables d'environnement

Le repo fournit le modele versionne :

```
$HOME/Cycle-Analyst-App/cycle-analyst.env.example
```

Le fichier reel utilise par l'app est local au Pi :

```
$HOME/Cycle-Analyst-App/cycle-analyst.env
```

Il est cree automatiquement si absent, en copiant `cycle-analyst.env.example`.

Il est charge au demarrage de l'app et par le service systemd. Ce fichier est ignore par Git, donc chaque Pi garde ses propres valeurs.

Important pour un Pi client : si `cycle-analyst.env` existe deja, verifie qu'il ne contient pas un ancien identifiant de vehicule. `MONITOR_DEVICE_ID` doit etre vide ou correspondre au hostname du Pi courant.

```
grep -n '^MONITOR_DEVICE_ID=' "$HOME/Cycle-Analyst-App/cycle-analyst.env"
hostname
```

Sur `sc-vehicule-5`, ces deux valeurs doivent etre coherentes :

```
MONITOR_DEVICE_ID=sc-vehicule-5
```

ou bien :

```
MONITOR_DEVICE_ID=
```

Si la valeur est vide, l'app utilise automatiquement le hostname du Pi.

Tu peux l'ajuster depuis l'interface :

```
http://<hostname>.local/settings
```

Exemple :

```
http://sc-vehicule-5.local/settings
```

Exemple de contenu pour le Pi client `sc-vehicule-5`. Pour un autre Pi, adapter au minimum `APP_PMTILES_PATH`. `MONITOR_DEVICE_ID` peut rester vide si le hostname du Pi est correct.

```
# Cycle Analyst App - editable app configuration

# Runtime local
APP_VAR_DIR=var

# A utiliser seulement pour un lancement monoprocess historique.
# Le setup recommande utilise plutot cycle-recorder.service.
# APP_START_READER=1
# En mode services separes, l'interface lit l'etat ecrit par le recorder.
APP_LIVE_STATE_FROM_FILES=1

# GPS USB NMEA. Les dongles VK-162 apparaissent souvent en /dev/ttyACM0.
APP_GPS_PORT=/dev/ttyACM0
APP_GPS_BAUDRATE=9600

# Carte offline PMTiles servie par /tiles/basemap.pmtiles.
APP_PMTILES_PATH=/home/danieldilg/Documents/tiles.pmtiles
```

```

# Estimation solaire / batterie.
# Si la position exacte n'est pas encore connue, laisse les valeurs par défaut
# ou remplace-les par la latitude/longitude du depart.
APP_SOLAR_PANEL_MAX_W=590
APP_SOLAR_LAT=48.8566
APP_SOLAR_LON=2.3522
APP_BATTERY_NOMINAL_VOLTAGE=48.1
APP_BATTERY_DISCHARGE_CURVE_FILE=var/battery_curve_lg_mh1_from_trip.json

# Capteur solaire INA228. Laisser commente si le Pi n'a pas ce capteur.
# APP_SOLAR_SENSOR=ina228
# APP_SOLAR_I2C_BUS=1
# APP_SOLAR_I2C_ADDR=0x45
# APP_SOLAR_SHUNT_OHMS=0.0002
# APP_SOLAR_MAX_AMPS=204.8
# APP_SOLAR_CURRENT_GAIN=1.0
# APP_SOLAR_CURRENT_OFFSET=0.0
# APP_SOLAR_CURRENT_DEADBAND_A=0.15
# Decommenter seulement si le courant solaire apparait avec le mauvais signe.
# APP_SOLAR_INVERT_SIGN=true

# Camera USB pour capture photo pendant les sessions.
# Sur Pi3, garder une capture courte pour eviter les timeouts.
# --palette YUYV evite les images grises observees avec le flux MJPEG de cette camera.
APP_CAMERA_COMMAND=fswebcam -d /dev/video0 -q -S 10 --palette YUYV -r 640x480 --jpeg 70 --no-banner {ou

# Synchronisation vers monitor_server. Laisser MONITOR_URL vide seulement pour desactiver.
# Vide = utilise automatiquement le hostname du Pi.
MONITOR_DEVICE_ID=
MONITOR_URL=http://91.134.243.157:8080
MONITOR_USER=
MONITOR_PASS=
MONITOR_UPLOAD_CHUNK_SIZE=1000

```

Si tu preferes garder le fichier dans `/etc/cycle-analyst`, tu peux copier le
fichier du repo puis lancer le service avec `APP_ENV_FILE` :

```

sudo install -d -m 0755 /etc/cycle-analyst
sudo cp "$HOME/Cycle-Analyst-App/cycle-analyst.env" /etc/cycle-analyst/cycle-analyst.env
sudo chown root:root /etc/cycle-analyst/cycle-analyst.env
sudo chmod 0640 /etc/cycle-analyst/cycle-analyst.env

```

9. Variables disponibles pour l'app embarquee

| Variable | Defaut dans le code | Usage |

| --- | --- | --- |

| `APP_VAR_DIR` | `var` | Dossier des DB, sessions, profils, photos locales et etat runtime. |

| `APP_START_READER` | `0` | Force le reader au moment de l'import `cycle_server`. Preferer `cycle-recorder.service` sur Raspberry Pi. |

| `APP_LIVE_STATE_FROM_FILES` | `1` | L'interface web relit les metriques live ecrites par le recorder. |

| `APP_SCHEDULE_PHOTOS` | `1` | Active les captures photo dans le process courant. Le setup split met `0` dans `cycle-recorder.service` et `1` dans `cycle-photo.service`. |

| `APP_PHOTO_WORKER_INTERVAL_SECONDS` | `1` | Frequence de lecture des metriques live par le worker photo. |

| `APP_PHOTO_UPLOAD_RETRY_SECONDS` | `15` | Delai entre deux tentatives d'envoi des photos en attente. |

APP_GPS_PORT	/dev/ttyACM0	Port serie du GPS NMEA.
APP_GPS_BAUDRATE	9600	Baudrate du GPS.
APP_PMTILES_PATH	vide	Fichier .pmtiles offline, par exemple /home/danieldilg/Documents/tiles.pmtiles.
APP_SOLAR_PANEL_MAX_W	590	Puissance max panneaux pour estimation solaire.
APP_SOLAR_LAT	48.8566	Latitude par default pour estimation solaire.
APP_SOLAR_LON	2.3522	Longitude par default pour estimation solaire.
APP_BATTERY_NOMINAL_VOLTAGE	48.1	Tension nominale batterie pour calcul Wh.
APP_BATTERY_DISCHARGE_CURVE_FILE	vide	Fichier JSON optionnel de courbe voltage/SOC.
APP_SOLAR_SENSOR	vide	Mettre ina228 pour activer le capteur solaire I2C.
APP_SOLAR_I2C_BUS	1	Bus I2C INA228.
APP_SOLAR_I2C_ADDR	0x45	Adresse INA228. 0 probe 0x41, 0x44, 0x45.
APP_SOLAR_SHUNT_OHMS	0.0002	Valeur du shunt.
APP_SOLAR_MAX_AMPS	204.8	Courant pleine echelle pour CURRENT_LSB.
APP_SOLAR_CURRENT_GAIN	1.0	Correction multiplicative courant.
APP_SOLAR_CURRENT_OFFSET	0.0	Offset courant en amperes.
APP_SOLAR_CURRENT_DEADBAND_A	0.15	Zone morte des petits courants.
APP_SOLAR_INVERT_SIGN	vide	true pour inverser le signe du courant solaire.
APP_CAMERA_COMMAND	auto	Commande custom camera. {output} est remplace par le fichier JPEG.
MONITOR_DEVICE_ID	hostname	ID unique envoye au monitor.
MONITOR_URL	http://91.134.243.157:8080	URL du monitor. Vide = sync desactivee.
MONITOR_USER	vide	Login Basic Auth monitor.
MONITOR_PASS	vide	Mot de passe Basic Auth monitor.
MONITOR_UPLOAD_CHUNK_SIZE	1000	Nombre de lignes envoyees par paquet au monitor.

10. Telecharger les tiles PMTiles pour la basemap vectorielle

L'app cherche la carte offline a l'emplacement configure par :

```
APP_PMTILES_PATH=/home/jeandard/Documents/tiles.pmtiles
```

Le frontend sert ensuite ce fichier avec l'URL interne /tiles/basemap.pmtiles.

Le style actuel est prevu pour une basemap vectorielle compatible Protomaps.

Le fichier planet complet Protomaps est tres gros. Il vaut mieux extraire une zone avec pmtiles extract, par exemple une region autour de l'itineraire.

References utiles :

- <https://docs.protomaps.com/guide/getting-started>

- <https://docs.protomaps.com/pmtiles/cli>
- <https://docs.protomaps.com/basemaps/downloads>

10.1 Choisir la zone a extraire

Il faut definir une bounding box au format :

```
MIN_LON, MIN_LAT, MAX_LON, MAX_LAT
```

Exemple :

```
BBOX="-6.0, 41.0, 10.0, 52.5"
```

Cette bbox couvre grossierement une grande partie de l'Europe de l'Ouest. Pour reduire le poids du fichier, utilise une bbox plus petite autour du parcours.

Tu peux trouver une bbox avec un outil comme :

```
https://bboxfinder.com
```

10.2 Choisir la source Protomaps

Aller sur :

```
https://maps.protomaps.com/builds
```

Choisir un build recent, puis copier son URL `.pmtiles`.

Exemple :

```
SOURCE_PMTILES_URL="https://build.protomaps.com/YYYYMMDD.pmtiles"
```

Remplace `YYYYMMDD` par le build choisi sur la page Protomaps.

10.3 Methode A - Telecharger directement depuis le Pi

Installer le CLI `pmtiles` sur le Pi.

Le plus simple est d'ouvrir la page suivante depuis un navigateur :

```
https://github.com/protomaps/go-pmtiles/releases/latest
```

Telecharger l'asset Linux ARM64 pour Raspberry Pi OS 64-bit, puis copier le binaire `pmtiles` dans `/usr/local/bin`.

Verifier :

```
pmtiles --help
```

Creer le dossier cible :

```
mkdir -p /home/jeandard/Documents
```

Extraire la zone :

```
SOURCE_PMTILES_URL="https://build.protomaps.com/YYYYMMDD.pmtiles"  
BBOX="-6.0,41.0,10.0,52.5"  
pmtiles extract "$SOURCE_PMTILES_URL" /home/jeandard/Documents/tiles.pmtiles --bbox="$BBOX"
```

Vérifier le fichier :

```
ls -lh /home/jeandard/Documents/tiles.pmtiles  
pmtiles show /home/jeandard/Documents/tiles.pmtiles
```

Redémarrer l'app pour être sûr que le chemin est pris en compte :

```
sudo systemctl restart cycle-analyst.service
```

Tester depuis le Pi :

```
curl -I http://127.0.0.1:5050/tiles/basemap.pmtiles
```

10.4 Méthode B - Télécharger sur un PC puis envoyer au Pi

Cette méthode est souvent plus confortable si le PC a une meilleure connexion.

Sur le PC, installer le CLI `pmtiles` depuis :

```
https://github.com/protomaps/go-pmtiles/releases/latest
```

Télécharger l'asset Windows AMD64, puis vérifier dans PowerShell :

```
pmtiles.exe --help
```

Depuis PowerShell, extraire la zone :

```
$SOURCE_PMTILES_URL = "https://build.protomaps.com/YYYYMMDD.pmtiles"  
$BBOX = "-6.0,41.0,10.0,52.5"  
.\pmtiles.exe extract $SOURCE_PMTILES_URL .\tiles.pmtiles --bbox=$BBOX
```

Vérifier :

```
Get-Item .\tiles.pmtiles  
.\pmtiles.exe show .\tiles.pmtiles
```

Envoyer le fichier vers le Pi 1 :

```
ssh jeandard@sc-vehicule-1.local "mkdir -p /home/jeandard/Documents"  
scp .\tiles.pmtiles jeandard@sc-vehicule-1.local:/home/jeandard/Documents/tiles.pmtiles
```

Pour le Pi 2 :

```
ssh jeandard@sc-vehicule-2.local "mkdir -p /home/jeandard/Documents"  
scp .\tiles.pmtiles jeandard@sc-vehicule-2.local:/home/jeandard/Documents/tiles.pmtiles
```

Puis sur le Pi :

```
ls -lh /home/jeandard/Documents/tiles.pmtiles
sudo systemctl restart cycle-analyst.service
curl -I http://127.0.0.1:5050/tiles/basemap.pmtiles
```

10.5 Remplacer les tiles plus tard

Pour mettre a jour la carte, remplace simplement :

```
/home/jeandard/Documents/tiles.pmtiles
```

Puis :

```
sudo systemctl restart cycle-analyst.service
```

Si le navigateur garde l'ancien rendu, recharge la page avec un refresh force.

11. Tester l'app manuellement

```
cd /home/jeandard/Cycle-Analyst-App
source .env/bin/activate
set -a
. /home/jeandard/Cycle-Analyst-App/cycle-analyst.env
set +a
python cycle_server.py
```

Le code lance Flask sur 0.0.0.0:5050.

Tester depuis le Pi dans un second terminal :

```
curl http://127.0.0.1:5050
curl http://127.0.0.1:5050/metrics
curl http://127.0.0.1:5050/gps_status
curl http://127.0.0.1:5050/sys_metrics
```

Arreter avec Ctrl+C.

12. Tester les peripheriques optionnels

Port Cycle Analyst

Le mode live lit par défaut /dev/ttyUSB0 a 9600 bauds via pyserial.

Voir les ports detectes :

```
ls -l /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
```

Si le Cycle Analyst n'est pas sur /dev/ttyUSB0, il faut modifier

VEHICLE_CONFIGS dans app/config.py ou creer une regle udev stable.

GPS

```
ls -l /dev/ttyACM* /dev/ttyUSB* 2>/dev/null
timeout 5 cat /dev/ttyACM0
```

Tu dois voir des lignes NMEA commençant par `$GPGGA`, `$GNGGA`, `$GPRMC` ou `$GNRMC`.

INA228 solaire

Vérifier le bus I2C :

```
i2cdetect -y 1
```

Debugger le capteur avec le script du repo :

```
cd /home/jeandard/Cycle-Analyst-App
source .venv/bin/activate
python scripts/ina228_debug.py --addr 0 --interval 1 --once
```

Si le courant est inverse, ajouter dans le fichier env :

```
APP_SOLAR_INVERT_SIGN=true
```

Camera

Vérifier que la camera USB est détectée :

```
lsusb
v4l2-ctl --list-devices
ls -l /dev/video*
```

Tester la capture avec la même commande que l'app :

```
fswebcam -d /dev/video0 -q -S 10 --palette YUYV -r 640x480 --jpeg 70 --no-banner /tmp/cycle-test.jpg
ls -lh /tmp/cycle-test.jpg
```

Si l'image est uniformément grise, vérifier que `--palette YUYV` est bien présent. Sur cette camera, le mode MJPEG peut produire une image grise même si la camera est correctement détectée.

Si la camera apparaît sur un autre device, par exemple `/dev/video1`, adapter `APP_CAMERA_COMMAND` dans `/home/jeandard/Cycle-Analyst-App/cycle-analyst.env` ou depuis `/settings`.

13. Installer les services systemd séparés

L'installation recommandée sépare l'enregistrement, les photos et l'interface :

- `cycle-recorder.service` lit le Cycle Analyst, le GPS et l'I2C, puis écrit les données SQLite.
- `cycle-photo.service` lit les métriques live du recorder, déclenche la caméra aux checkpoints de distance, garde les photos en attente dans `var/pending_photos`, puis retente l'upload.
- `cycle-analyst.service` sert uniquement l'interface web sur le port 5050.
- `nginx` expose l'interface sur le port 80.

Cette séparation permet à l'enregistrement de continuer même si le dashboard, nginx, le navigateur, la caméra ou l'upload photo/reseau se bloquent.

Depuis le repo :

```
cd "$HOME/Cycle-Analyst-App"
python3 scripts/setup_pi_services.py
```

Le premier lancement est un aperçu. Pour installer ou mettre à jour les services :

```
python3 scripts/setup_pi_services.py --apply
```

Par défaut, le script utilise :

- le repo courant comme `WorkingDirectory`
- l'utilisateur courant comme `User` et `Group`
- le hostname courant pour nginx : `<hostname>.local <hostname>`

Sur le Pi client `danieldilg@sc-vehicule-5`, lance donc simplement depuis `/home/danieldilg/Cycle-Analyst-App` :

```
python3 scripts/setup_pi_services.py --apply
```

Spécifier le nom local du véhicule seulement si le hostname du Pi n'est pas celui voulu :

```
python3 scripts/setup_pi_services.py --apply --server-name "sc-vehicule-5.local sc-vehicule-5"
```

Vérifier :

```
systemctl status cycle-recorder.service cycle-photo.service cycle-analyst.service --no-pager
curl http://127.0.0.1:5050/metrics
curl http://127.0.0.1/
```

Voir les logs :

```
journalctl -u cycle-recorder.service -f
journalctl -u cycle-photo.service -f
journalctl -u cycle-analyst.service -f
sudo tail -f /var/log/nginx/error.log
```

Par défaut, le service recorder installé par ce setup désactive la capture photo dans le processus d'enregistrement (`APP_SCHEDULE_PHOTOS=0`). C'est volontaire :

sur Pi3, la camera et l'upload ne doivent pas pouvoir bloquer l'écriture des données. Le service `cycle-photo.service` prend le relais pour les photos.

14. Autoriser le bouton de redemarrage de l'UI

L'endpoint `/restart_service` exécute :

```
sudo systemctl restart cycle-analyst.service
```

En installation séparée, ce bouton redémarre seulement l'interface web. Il ne redémarre pas `cycle-recorder.service`, pour éviter d'interrompre l'enregistrement. Sans règle sudoers, le bouton de l'interface ne peut pas redémarrer le service web.

Créer une règle dédiée :

```
sudo visudo -f /etc/sudoers.d/cycle-analyst
```

Contenu :

```
<utilisateur> ALL=NOPASSWD: /bin/systemctl restart cycle-analyst.service
<utilisateur> ALL=NOPASSWD: /usr/bin/systemctl restart cycle-analyst.service
```

Exemple pour le Pi client :

```
danielgilg ALL=NOPASSWD: /bin/systemctl restart cycle-analyst.service
danielgilg ALL=NOPASSWD: /usr/bin/systemctl restart cycle-analyst.service
```

Vérifier :

```
sudo -l -U "$USER"
```

15. Configurer nginx pour l'URL locale

Le script `scripts/setup_pi_services.py --apply` configure déjà nginx. Cette section sert seulement si tu veux le faire à la main.

Créer le site :

```
sudo nano /etc/nginx/sites-available/cycle-analyst
```

Contenu pour le Pi courant :

```
server {
    listen 80;
    server_name sc-vehicule-5.local sc-vehicule-5;

    location / {
        proxy_pass http://127.0.0.1:5050;
        proxy_http_version 1.1;
    }
}
```

```

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

Activer :

```

sudo ln -sf /etc/nginx/sites-available/cycle-analyst /etc/nginx/sites-enabled/cycle-analyst
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl enable --now nginx
sudo systemctl reload nginx

```

Tester depuis le Pi :

```
curl http://127.0.0.1
```

Tester depuis un autre appareil du meme reseau :

```
curl http://sc-vehicule-5.local
```

Ou ouvrir :

```
http://sc-vehicule-5.local
```

16. Verifier le mDNS .local

Sur le Pi :

```

avahi-resolve-host-name "$(hostname).local"
hostname -I

```

Depuis le Mac ou PC :

```

ping sc-vehicule-5.local
ssh danieldilg@sc-vehicule-5.local

```

Important : certains hotspots telephone bloquent le multicast ou l'isolation client. Dans ce cas, l'IP peut fonctionner alors que .local ne se resout pas.

17. Option : creer un hotspot Wi-Fi de secours

Cette option garde la connexion au telephone comme reseau principal, mais cree aussi un hotspot Wi-Fi du Pi. Si le Pi ne rejoint pas le telephone au boot, NetworkManager peut activer ce hotspot de secours.

Limite importante : avec un seul Wi-Fi wlan0, le Pi ne peut pas etre connecte au telephone et servir son propre hotspot en meme temps de maniere fiable. Un seul profil sera actif a la fois.

Connexion telephone prioritaire

Remplacer le SSID et le mot de passe par ceux du partage de connexion :

```
sudo nmcli con add type wifi ifname wlan0 con-name "iPhone de Jean" ssid "iPhone de Jean"
sudo nmcli con modify "iPhone de Jean" \
  wifi-sec.key-mgmt wpa-psk \
  wifi-sec.psk "MOT_DE_PASSE_TELEPHONE" \
  ipv4.method auto \
  ipv6.method disabled \
  connection.autoconnect yes \
  connection.autoconnect-priority 999
```

Si le profil existe deja :

```
sudo nmcli con modify "iPhone de Jean" \
  ipv4.method auto \
  ipv6.method disabled \
  connection.autoconnect yes \
  connection.autoconnect-priority 999
```

Hotspot du Pi en secours

Adapter le nom au vehicule :

- Pi 1 : SC-Vehicule-1
- Pi 2 : SC-Vehicule-2
- Pi 3 : SC-Vehicule-3

Exemple pour le Pi 2 :

```
sudo nmcli con add type wifi ifname wlan0 con-name SC-Vehicule-2 ssid SC-Vehicule-2
sudo nmcli con modify SC-Vehicule-2 \
  802-11-wireless.mode ap \
  802-11-wireless.band bg \
  wifi-sec.key-mgmt wpa-psk \
  wifi-sec.psk "supercyclepass!" \
  ipv4.method shared \
  ipv6.method ignore \
  connection.autoconnect yes \
  connection.autoconnect-priority -10
sudo nmcli dev set wlan0 managed yes
sudo systemctl restart NetworkManager
```

`802-11-wireless.band bg` force un reseau 2.4 GHz, souvent plus visible et plus

stable avec les telephones et les Raspberry Pi.

Verification

```
nmcli con show
nmcli -f NAME,TYPE,AUTOCONNECT,AUTOCONNECT-PRIORITY con show
nmcli con show --active
```

Etat attendu quand le telephone est disponible :

iPhone de Jean	wifi	yes	999
SC-Vehicule-2	wifi	yes	-10

Et dans les connexions actives :

iPhone de Jean	wifi	wlan0
----------------	------	-------

Si le Pi active son hotspot, se connecter au Wi-Fi `SC-Vehicule-2`, puis ouvrir
ou joindre le Pi avec :

```
http://10.42.0.1
ssh jeandard@10.42.0.1
```

Basculer manuellement

Forcer le hotspot du Pi :

```
sudo nmcli con down "iPhone de Jean" || true
sudo nmcli con up SC-Vehicule-2
```

Revenir au telephone :

```
sudo nmcli con down SC-Vehicule-2 || true
sudo nmcli con up "iPhone de Jean"
```

Nettoyer les doublons

Si plusieurs profils ont le meme nom :

```
nmcli -f NAME,UUID,TYPE,AUTOCONNECT,DEVICE con show
```

Supprimer les doublons non actifs avec leur UUID :

```
sudo nmcli con delete uuid UUID_A_SUPPRIMER
```

Garder de preference le profil qui affiche `wlan0` dans la colonne `DEVICE`.

18. Config SSH pour VS Code

Sur ton Mac ou PC, editer `~/.ssh/config` :

```
Host sc1 sc-vehicule-1.local
  HostName sc-vehicule-1.local
  User jeandard
  Port 22
  IdentityFile ~/.ssh/id_ed25519
  IdentitiesOnly yes
  ServerAliveInterval 30
  ServerAliveCountMax 6

Host sc2 sc-vehicule-2.local
  HostName sc-vehicule-2.local
  User jeandard
  Port 22
```

```
IdentityFile ~/.ssh/id_ed25519
IdentitiesOnly yes
ServerAliveInterval 30
ServerAliveCountMax 6
```

Ensuite dans VS Code :

- Remote-SSH: Connect to Host...
- choisir sc1 ou sc2

19. Mise a jour du repo sur un Pi deja installe

```
cd "$HOME/Cycle-Analyst-App"
git status
git pull --ff-only
source .venv/bin/activate
python -m pip install -r requirements.txt
sudo systemctl restart cycle-recorder.service cycle-photo.service cycle-analyst.service
sudo systemctl status cycle-recorder.service cycle-photo.service cycle-analyst.service --no-pager
```

Si tu modifies `cycle-analyst.env` :

```
sudo systemctl restart cycle-analyst.service
```

Si tu modifies le fichier `.service` :

```
sudo systemctl daemon-reload
sudo systemctl restart cycle-analyst.service
```

20. Option : installer monitor_server

Le `monitor_server` peut tourner sur un serveur distant, un laptop ou un Pi.

Il recoit les uploads des vehicules quand `MONITOR_URL` est configure cote app embarquee.

Si tu veux le lancer sur le meme repo :

```
cd "$HOME/Cycle-Analyst-App"
source .venv/bin/activate
MONITOR_USER=jean MONITOR_PASS=oklm python monitor_server/app.py
```

Par default il ecoute sur `0.0.0.0:8080`.

Variables du monitor_server

Variable	Defaut	Usage
----------	--------	-------

---	---	---
-----	-----	-----

<code>MONITOR_PORT</code>	<code>8080</code>	Port HTTP du monitor.
---------------------------	-------------------	-----------------------

<code>MONITOR_DB</code>	<code>monitor_server/monitor.db</code>	DB SQLite du monitor.
-------------------------	--	-----------------------

<code>MONITOR_MEDIA_DIR</code>	<code>monitor_server/media</code>	Stockage des photos recues.
--------------------------------	-----------------------------------	-----------------------------

MONITOR_USER	vide	Login Basic Auth requis pour les APIs protegees.
MONITOR_PASS	vide	Mot de passe Basic Auth.
MONITOR_TERRAIN_RESOURCE	ign_rge_alti_wld	Ressource altimetrie IGN.
MONITOR_TERRAIN_CACHE_DECIMALS	5	Precision cache altitude.
MONITOR_TERRAIN_BATCH_SIZE	5000	Taille batch IGN.
MONITOR_TERRAIN_TIMEOUT_SEC	8	Timeout IGN.
MONITOR_TERRAIN_FALLBACK_DATASET	srtm30m	Dataset OpenTopoData fallback.
MONITOR_TERRAIN_FALLBACK_API_URL	https://api.opentopodata.org/v1/srtm30m	URL fallback altitude.
MONITOR_TERRAIN_FALLBACK_BATCH_SIZE	100	Taille batch fallback.
MONITOR_TERRAIN_FALLBACK_THROTTLE_SEC	1.0	Pause entre batchs fallback.
MONITOR_TERRAIN_BACKFILL_LIMIT_POINTS	500	Points max par backfill altitude.

Service systemd optionnel du monitor

Creer un fichier env separe :

```
sudo nano /etc/cycle-analyst/monitor.env
```

Exemple :

```
MONITOR_PORT=8080
MONITOR_DB=/home/jeandard/Cycle-Analyst-App/var/monitor.db
MONITOR_MEDIA_DIR=/home/jeandard/Cycle-Analyst-App/var/monitor_media
MONITOR_USER=jean
MONITOR_PASS=oklm
MONITOR_TERRAIN_RESOURCE=ign_rge_alti_wld
MONITOR_TERRAIN_CACHE_DECIMALS=5
MONITOR_TERRAIN_BATCH_SIZE=5000
MONITOR_TERRAIN_TIMEOUT_SEC=8
MONITOR_TERRAIN_FALLBACK_DATASET=srtm30m
MONITOR_TERRAIN_FALLBACK_BATCH_SIZE=100
MONITOR_TERRAIN_FALLBACK_THROTTLE_SEC=1.0
MONITOR_TERRAIN_BACKFILL_LIMIT_POINTS=500
```

Proteger :

```
sudo chown root:root /etc/cycle-analyst/monitor.env
sudo chmod 0640 /etc/cycle-analyst/monitor.env
```

Creer le service :

```
sudo nano /etc/systemd/system/cycle-monitor.service
```

Contenu :

```
[Unit]
Description=Cycle Monitor Server
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
```

```
User=jeandard
Group=jeandard
WorkingDirectory=/home/jeandard/Cycle-Analyst-App
EnvironmentFile=-/etc/cycle-analyst/monitor.env
Environment=PYTHONUNBUFFERED=1
Environment=PATH=/home/jeandard/Cycle-Analyst-App/.venv/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
ExecStart=/home/jeandard/Cycle-Analyst-App/.venv/bin/gunicorn -c /home/jeandard/Cycle-Analyst-App/monitor.conf
Restart=always
RestartSec=5
KillSignal=SIGINT
TimeoutStopSec=20

[Install]
WantedBy=multi-user.target
```

Activer :

```
sudo systemctl daemon-reload
sudo systemctl enable --now cycle-monitor.service
sudo systemctl status cycle-monitor.service --no-pager
```

Tester :

```
curl http://127.0.0.1:8080
```

21. Brancher un Pi vehicule au monitor

Dans `$HOME/Cycle-Analyst-App/cycle-analyst.env` sur chaque Pi vehicule :

```
MONITOR_DEVICE_ID=sc-vehicule-5
MONITOR_URL=http://91.134.243.157:8080
MONITOR_USER=jean
MONITOR_PASS=oklm
```

Puis :

```
sudo systemctl restart cycle-analyst.service
journalctl -u cycle-analyst.service -f
```

La sync tourne toutes les 60 secondes quand `MONITOR_URL` est defini.

22. Checklist finale par Pi

- hostname unique OK
- SSH OK
- `avahi-daemon` actif
- utilisateur dans les groupes `dialout`, `i2c`, `video`
- repo clone dans `/home/<utilisateur>/Cycle-Analyst-App`
- acces GitHub OK : token HTTPS client ou cle SSH/deploy key
- `.venv` cree
- dependances Python installees
- `cycle-analyst.env` present localement sur le Pi

- APP_VAR_DIR pointe vers le var/ du repo
- /home/<utilisateur>/Documents/tiles.pmtiles existe si la basemap offline est utilisee
- MONITOR_DEVICE_ID unique
- cycle-recorder.service actif
- cycle-photo.service actif
- cycle-analyst.service actif
- nginx actif
- curl http://127.0.0.1:5050/metrics repond sur le Pi
- http://sc-vehicule-X.local repond depuis un autre appareil
- si solaire : i2cdetect -y 1 et scripts/ina228_debug.py OK
- si GPS : /gps_status recoit des donnees
- si camera : capture test OK
- si monitor : heartbeat visible cote monitor

23. Depannage

Le service ne demarre pas

```
sudo systemctl status cycle-analyst.service --no-pager
journalctl -u cycle-analyst.service -n 100 --no-pager
sudo systemctl status cycle-recorder.service --no-pager
journalctl -u cycle-recorder.service -n 100 --no-pager
sudo systemctl status cycle-photo.service --no-pager
journalctl -u cycle-photo.service -n 100 --no-pager
```

Verifier :

- chemin du repo
- chemin du venv
- syntaxe de cycle-analyst.env
- permissions sur les ports serie/I2C/camera

La page web locale marche, mais pas .local

```
systemctl status avahi-daemon --no-pager
hostname
hostname -I
avahi-resolve-host-name sc-vehicule-1.local
```

Si l'IP marche mais pas .local, le reseau bloque probablement le mDNS.

Le Cycle Analyst ne donne pas de donnees

```
ls -l /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
groups
sudo systemctl status cycle-recorder.service --no-pager
journalctl -u cycle-recorder.service -n 100 --no-pager
```

```
sudo fuser -v /dev/ttyUSB0 /dev/ttyACM0 2>/dev/null || true
```

Vérifier que l'utilisateur est dans `dialout` et que le port live du code est bien `/dev/ttyUSB0`.

Si la page `/start` affiche `Connection: Inactive` alors que `cycle-recorder.service` tourne, vérifier que le repo contient bien le correctif qui publie les snapshots live même avant le début d'une session :

```
cd /home/jeandard/Cycle-Analyst-App
grep -n "last_live_state_write_time" app/reader.py
SID=$(cat var/current_session.txt 2>/dev/null)
ls -lh "var/session_metrics/${SID}_session_metrics.json" 2>/dev/null
curl -s http://127.0.0.1:5050/metrics | head -c 300
```

Le `grep` doit afficher une ligne et le fichier de métriques doit avoir une date récente quand le Cycle Analyst est branché. Sinon :

```
git pull
sudo systemctl restart cycle-recorder.service cycle-photo.service cycle-analyst.service
```

Le GPS ne donne pas de fix

```
curl http://127.0.0.1:5050/gps_status
timeout 5 cat /dev/ttyACM0
```

Adapter `APP_GPS_PORT` si le GPS est sur un autre port.

Le capteur INA228 ne répond pas

```
sudo raspi-config nonint do_i2c 0
i2cdetect -y 1
cd /home/jeandard/Cycle-Analyst-App
source .venv/bin/activate
python scripts/ina228_debug.py --addr 0 --once
```

Si aucune adresse `0x41`, `0x44` ou `0x45` n'apparaît, vérifier le câblage et l'alimentation du capteur.

Si seul `cycle-recorder.service` devient instable alors que les services web et photo restent actifs, vérifier que le recorder n'accumule pas les ouvertures du bus I2C :

```
PID=$(systemctl show -p MainPID --value cycle-recorder.service)
ls -l /proc/$PID/fd 2>/dev/null | grep -c '/dev/i2c-1'
ls -l /proc/$PID/fd 2>/dev/null | grep -E '/dev/ttyUSB|/dev/ttyACM|/dev/i2c'
```

Le nombre de `/dev/i2c-1` doit rester très bas. Si le capteur INA228 ne répond plus pendant un trajet, désactiver temporairement le solaire pour protéger l'enregistrement CA/GPS :

```
cd /home/jeandard/Cycle-Analyst-App
cp cycle-analyst.env cycle-analyst.env.no-solar.$(date +%H%M%S)
grep -q '^APP_SOLAR_SENSOR=' cycle-analyst.env \
  && sed -i 's/^APP_SOLAR_SENSOR=.*\/APP_SOLAR_SENSOR=/' cycle-analyst.env \
  || echo 'APP_SOLAR_SENSOR=' >> cycle-analyst.env
sudo systemctl restart cycle-recorder.service
```

Le bouton restart de l'interface echoue

```
sudo -l -U jeandard
sudo systemctl restart cycle-analyst.service
```

Verifier `/etc/sudoers.d/cycle-analyst`.

VS Code Remote-SSH bloque

Sur le Pi :

```
rm -rf ~/.vscode-server ~/.vscode-remote
```

Puis retenter la connexion depuis VS Code.

Changement de cle SSH dans `known_hosts`

Sur le Mac ou PC :

```
ssh-keygen -R sc-vehicule-1.local
ssh-keygen -R sc-vehicule-2.local
ssh-keygen -R cycle.local
```

24. Resume ultra-court

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y git python3 python3-venv python3-pip nginx avahi-daemon curl i2c-tools
sudo usermod -aG dialout,i2c,video "$USER"
sudo raspi-config nonint do_i2c 0
sudo reboot
```

```
git config --global credential.helper store
cd "$HOME"
git clone https://github.com/regenbox-team/Cycle-Analyst-App.git Cycle-Analyst-App
cd Cycle-Analyst-App
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
mkdir -p var var/session_metrics var/live_photo var/pending_photos
mkdir -p "$HOME/Documents"
```

Puis ajuster/creer :

- `$HOME/Cycle-Analyst-App/cycle-analyst.env` cree automatiquement par l'app ou copie depuis `cycle-analyst.env.example`
- services et nginx via `python3 scripts/setup_pi_services.py --apply`

- `/etc/systemd/system/cycle-recorder.service`
- `/etc/systemd/system/cycle-photo.service`
- `/etc/systemd/system/cycle-analyst.service`
- `/etc/nginx/sites-available/cycle-analyst`
- `/etc/sudoers.d/cycle-analyst` si tu veux le bouton restart

Et activer :

```
python3 scripts/setup_pi_services.py --apply  
sudo nginx -t
```